EXPERIMENT 1

ER Modelling of a Campus Event Management System

1. Aim

To construct an Entity-Relationship model for a campus event portal that records students, event organizers, venues, events, registrations, payments, feedback, and event tags.

2. Problem Description

A university conducts workshops, cultural programs, technical talks, and competitions. The portal must store event details, show which organizer manages an event, reserve a venue, track student registrations, record payment status when an event has a fee, and collect feedback after an event. The model should reduce duplicated data and preserve valid links between records.

3. Theory

An Entity-Relationship (ER) model is a high-level blueprint for a database. It describes the main objects a system stores, the facts recorded about each object, and the rules that connect those objects. It is normally designed before tables are created.

Entity: A distinguishable object such as STUDENT, EVENT, or VENUE. **Attribute:** A property such as StudentEmail or EventDate. **Primary key:** An attribute, or a set of attributes, that uniquely identifies a record. **Foreign key:** A reference to a key in another table that keeps related records consistent.

Relationship and cardinality: A relationship expresses how entities are associated. One organizer may coordinate many events (1:N), and one venue may host many events over time (1:N). Students can attend many events and each event can have many students, so STUDENT and CAMPUS_EVENT have an M:N relationship resolved through REGISTRATION.

Associative entity: REGISTRATION stores details of the student-event relationship, including the registration timestamp and attendance status. **Multivalued attribute:** An event can have several tags, so tags are stored as separate EVENT_TAG rows rather than a comma-separated field. **Participation:** Every registration must refer to one existing student and one existing event; an event may not yet have any registrations.

4. Entities and Important Attributes

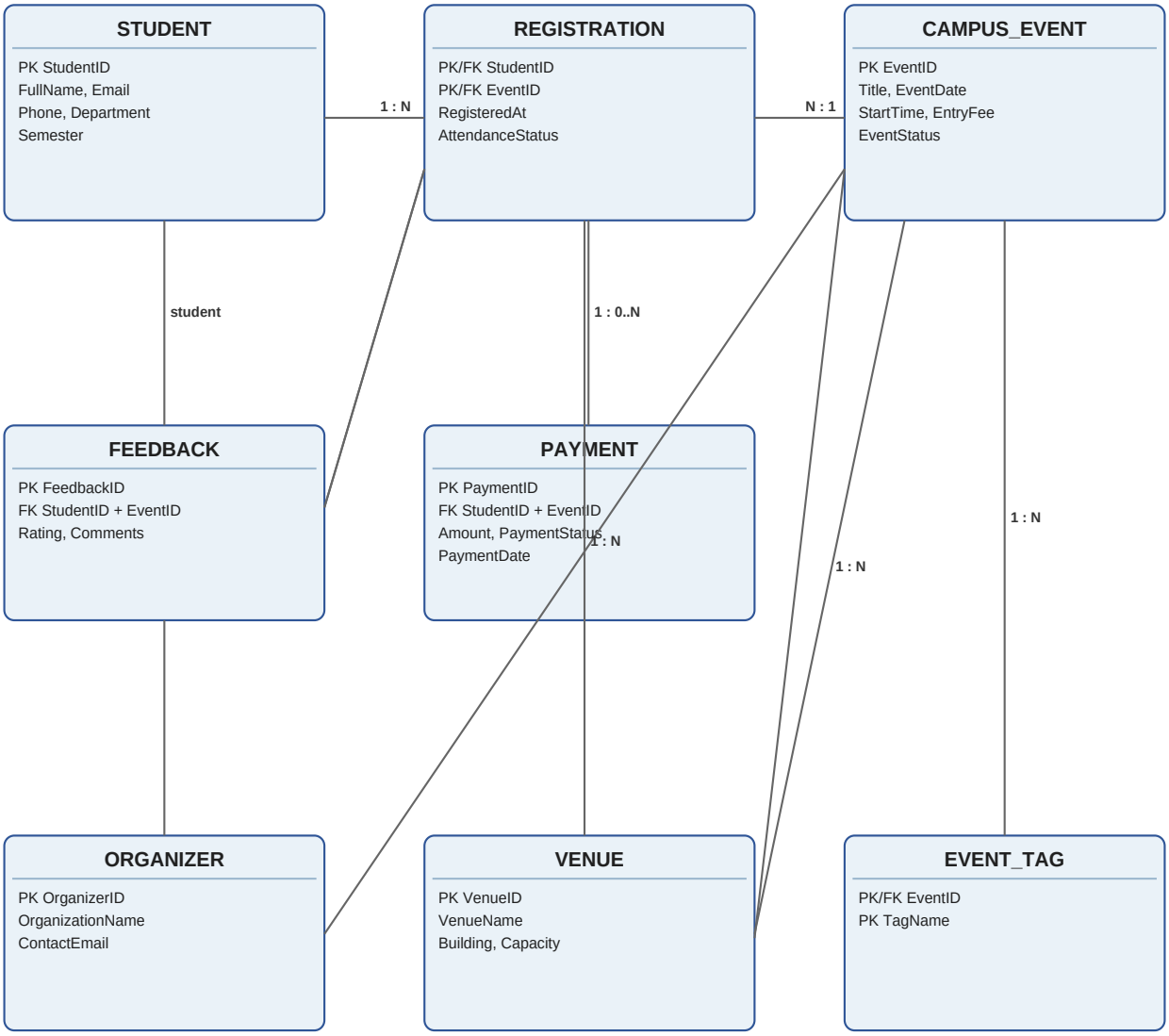
Entity	Key attribute	Other attributes / purpose
STUDENT	StudentID (PK)	FullName, Email, Phone, Department, Semester
ORGANIZER	OrganizerID (PK)	OrganizationName, ContactEmail
VENUE	VenueID (PK)	VenueName, Building, Capacity
CAMPUS_EVENT	EventID (PK)	Title, EventDate, StartTime, EntryFee, EventStatus, OrganizerID (FK), VenueID (FK)
REGISTRATION	(StudentID, EventID) (PK)	RegisteredAt, AttendanceStatus; joins students and events
PAYMENT	PaymentID (PK)	StudentID, EventID, Amount, PaymentStatus, PaymentDate
FEEDBACK	FeedbackID (PK)	StudentID, EventID, Rating, Comments
EVENT_TAG	(EventID, TagName) (PK)	Stores multiple labels for an event

5. Relationship Rules

• ORGANIZER 1 — N CAMPUS_EVENT: one organizer can manage multiple events; an event has at most one assigned organizer.
• VENUE 1 — N CAMPUS_EVENT: a venue can be used by multiple events on different dates; an event has at most one venue.
• STUDENT M — N CAMPUS_EVENT through REGISTRATION.
• REGISTRATION 1 — 0..N PAYMENT and REGISTRATION 1 — 0..1 FEEDBACK in this implementation model.
• CAMPUS_EVENT 1 — N EVENT_TAG.

6. ER Diagram

The diagram below presents the entities and the main relationship paths used by the proposed database.



PK = Primary Key FK = Foreign Key

REGISTRATION resolves the many-to-many relationship between STUDENT and CAMPUS_EVENT.

Note: This is a logical ER model. The relational implementation in Experiment 2 specifies the key and deletion rules in MySQL.

7. Result

The ER model for a Campus Event Management System has been prepared with entities, key attributes, relationship cardinalities, and an associative entity for student registrations.

8. Conclusion

The model provides a structured foundation for implementing a campus event database. Separating event tags, registrations, payments, and feedback helps avoid repeating the same information and makes relationships easier to validate.